Lab 1 — Teacher Understanding (Deck B)

Private prep deck — do not project to students.
Language: Roman Urdu explanations + English terms.

Goal: aap confidently padhao, hints do, marking fair rakho — bina over-spoiling ke.

Slide 1 — Lab 1 in one page

Students ko **stick** hona chahiye:

1. C# build pipeline (compiler vs CLR)
2. Value vs reference (array assign)
3. Data / field / record / file
4. Array ki 6 ops + insert/delete O(n) intuition
5. Debugger se logic bug nikalna
6. HashSet = memory dekar time bachana

Agar yeh 6 clear hain → Lab 2/Quiz 1 soft landing.

Slide 2 — CLO map (marks kahan se aate hain)

Lab Task /10 (course outline se mirror):

Component	CLO	Marks
Demo (run + debug)	CLO-2	2
Task completion	CLO-2	3
Code quality + AI disclosure	CLO-3	2
Viva / concepts	CLO-1	3

Assignment 1 (Labs 1–4) alag; Quiz 1 = Labs 1–3 concepts.

Slide 3 — SDK vs Runtime (30-second answer)

- **SDK:** develop/build ke tools (`dotnet build` , templates).
- **Runtime:** program **chalane** ka engine (CLR + libraries).

Student confusion: `dotnet` not recognized → PATH / terminal reopen.
Aap ka line: “Install ke baad terminal band karke dubara kholo.”

Slide 4 — Implicit usings

.NET 8 projects aksar `System` , `IO` , `Linq` , `Collections.Generic` auto-import.

Students sochenge “using likhe baghair `Console` kaise?” — yeh feature hai, magic nahi.

`System.Diagnostics` (`Stopwatch`) aur `System.Globalization` **explicit** `using` mangte hain.

Slide 5 — Value vs reference (heap picture)

Board pe draw karo:

```
int a=10; int b=a; b=99;     → do alag boxes

int[] first = {...};
int[] second = first;        → dono arrows SAME array
second[0]=99;                → first[0] bhi 99

Clone()                      → NEW array, alag arrow
```

Key sentence: “Reference copy = address copy, data copy nahi.”

Slide 6 — String special case

`string` reference type hai lekin **immutable**.

`s2 += "!"` naya object banata hai; `s1` purana rehta hai.

Students ko “array jaisa share” expect karna band karo — string alag rules.

Slide 7 — Insert / Delete: naya array kyun?

C# `int[]` fixed length.

Insert/Delete = naya size allocate + elements copy/shift → **O(n)**.

Isi liye baad mein `List<T>` aayega — abhi arrays only (Task 5 rule).

Slide 8 — Merge trap

`Merge` **sorted inputs** assume karta hai.

Agar student unsorted merge kare → output galat, algorithm “toot” nahi — **precondition** toot.

Viva line: “Merge sorted collections ko combine karta hai in O(n+m).”

Slide 9 — Stopwatch: Release mode

Debug build slow / noisy ho sakta hai.

Fair timing: `dotnet run -c Release`

Task 6: teen runs, **median** rakho — outliers avoid.

Slide 10 — Task 4: three bugs (deep dive)

`BuggyStats` with `{4,5,6,7}` and `{-7,-3,-9}` :

Bug	Where	Kind	Why

1	`i <= values.Length`	**Runtime**	Last index invalid → `IndexOutOfRangeException`
2	`total / values.Length`	**Logic**	int/int truncates → average 5 not 5.5
3	`int max = 0`	**Logic**	All-negative array → wrong max 0 instead of -3

Fix direction (aap ke notes):

1. `i < values.Length`
2. Cast: `(double)total / values.Length`
3. `max = values[0]` (empty check later courses mein)

Slide 11 — Hints ladder (bina full spoiler)

Task 4 help order:

1. "Expected output likho. Actual kya hai?"
2. "Kon sa test pe crash / wrong?"
3. "Breakpoint: loop last index kya hai?"
4. "Average fractional kyun nahi aa raha?"
5. "Negative-only array pe max seed kaun sa hona chahiye?"

Screenshot rule: Locals mein faulty variable dikhe.

Slide 12 — Task 5 marking checklist

- Arrays only — `List<T>` = marks cut (course rule)
- Menu 0–6 works
- `TryParse` + no crash on bad input
- Duplicate ID reject on enrol
- Remove by ID (not only by index)
- XML comments on methods student wrote
- Reuse `ArrayOperations` style

Minimum pass path: Display + Search + Enrol + Remove + validation.

Slide 13 — Task 6 coaching

- Worst-case arrays: no duplicates (`ids[i] = i * 3`)
- n = 5k, 10k, 20k, 40k
- Nested ~ $O(n^2)$, HashSet ~ O(n) build + O(1) checks
- Conclusion must answer: (a) cloud (b) edge device + extra space name

CLO-3: AI disclosure block — "None" bhi valid agar sach ho.

Slide 14 — Common errors → quick fix

Message	Fix
`dotnet` not recognised	Reinstall/PATH; reopen terminal
CS0103	Typo / case / missing using

CS5001	No `Main` — use `--use-program-main` pattern
FormatException	Use `TryParse`
IndexOutOfRange	Bound check `< Length`

Slide 15 — Weak students: minimum path

Must finish today:

- Task 1 screenshot + 2 short answers
- Task 2 profile card
- Task 3 Part A table + Part B/C written
- Task 4 corrected class + 2 test outputs + debugger shots

Tasks 5–6 → homework OK if time khatam.

Slide 16 — Strong students: stretch

- Task 5 full menu + merge
- Task 6 table + 6–8 line conclusion
- Post-lab Q3: comparison counts (Lab 2 bridge)

Unhe ask: “Best / middle / worst comparisons ka pattern kya hai?”

Slide 17 — Integrity / AI script (bolne layak)

“AI allowed **sirf** jahan instructor permit kare.
Use kiya to disclosure likho: tool, kya manga, kya change kiya, kaise verify kiya.
Hide karna = academic integrity case.
Viva mein har line explain karni hogi.”

Slide 18 — Exit checklist (aap ke liye)

- [ ] Attendance
- [ ] Tasks 1–4 progress noted
- [ ] Weak group ko homework clear
- [ ] AI disclosure remind
- [ ] Next lab: Big-O / Stopwatch reading
- [ ] Collect / LMS deadline announce

Quick analogies (pocket)

- **CLR** = restaurant kitchen jo recipe (IL) se dish (machine code) banati hai jab order aaye (JIT).
- **Reference** = do logon ke paas same ghar ka address.
- **HashSet** = vector index (josh) vs har page scan (linear).